

Rapport de Projet de Fin d'Études
En vue de l'obtention de la
Licence en Mathématiques et Informatique (LMI)
Spécialité : Informatique

DocuSense

**Plateforme Intelligente de Génération
de Documentation Technique Gamifiée**

Propulsée par l'Intelligence Artificielle Générative

Réalisé par : **Boumehdi Rym**
Encadrant : **[Nom de l'encadrant]**
Année universitaire : **2025 – 2026**

Dédicace

*À mes parents, pour leur soutien indéfectible et
leurs sacrifices*

*À mes enseignants, pour la transmission du sa-
voir*

*À tous ceux qui croient que la technologie peut
rendre
la connaissance accessible à tous.*

Boumehdi Rym

Remerciements

Avant tout, je remercie **Allah** de m'avoir accordé la santé, la volonté et la persévérance nécessaires pour mener à bien ce projet de fin d'études.

Je tiens à exprimer ma profonde gratitude à mon **encadrant**, dont les conseils avisés, la disponibilité et l'accompagnement rigoureux ont été déterminants dans la réalisation de ce travail. Ses orientations m'ont permis de structurer ma réflexion et d'approfondir mes connaissances tout au long de ce projet.

Mes remerciements s'adressent également à l'ensemble du **corps enseignant du département Informatique de l'IFAG** pour la qualité de la formation dispensée au cours de ces trois années de licence.

Je remercie les membres du **jury** d'avoir accepté d'évaluer ce travail.

Enfin, je remercie ma **famille** et mes **amis** pour leur soutien moral, leur patience et leurs encouragements tout au long de mon parcours académique.

Boumehdi Rym

Résumé

Résumé – Français

La documentation technique constitue un défi majeur pour les développeurs logiciels : elle est chronophage, souvent négligée et difficile à maintenir. Ce projet de fin d'études présente **DocuSense**, une plateforme web intelligente qui automatise la création de manuels utilisateurs grâce à l'intelligence artificielle générative.

DocuSense exploite l'API **Google Gemini 2.0 Flash** pour analyser des captures d'écran et des vidéos d'interfaces applicatives, puis générer automatiquement des étapes de documentation structurées. La plateforme intègre un **système de gamification** composé de quiz interactifs, de badges de progression (Bronze, Silver, Gold) et d'un classement compétitif, afin d'améliorer l'engagement et la rétention des connaissances par les utilisateurs finaux.

Techniquement, l'application repose sur une architecture avec React.js pour le frontend, Node.js/Express.js pour le backend, MySQL via Sequelize ORM pour la persistance des données, et une authentification sécurisée par JSON Web Tokens (JWT).

Mots-clés : Documentation automatique, Intelligence Artificielle Générative, Gemini AI, Gamification, JWT.

Abstract – English

Technical documentation represents a major challenge for software developers : it is time-consuming, often neglected, and difficult to maintain. This final year project presents **DocuSense**, an intelligent web platform that automates the creation of user manuals through generative artificial intelligence.

DocuSense leverages the **Google Gemini 2.0 Flash API** to analyze screenshots and videos of application interfaces, then automatically generates structured documentation steps. The platform integrates a **gamification system** consisting of interactive quizzes, progress badges (Bronze, Silver, Gold), and a competitive leaderboard, in order to improve engagement and knowledge retention by end users.

Technically, the application relies on React.js for the frontend, Node.js/Express.js for the backend, MySQL via Sequelize ORM for data persistence, and secure authentication via JSON Web Tokens (JWT).

Keywords : Automatic Documentation, Generative Artificial Intelligence, Gemini AI, Gamification, JWT.

ملخص — Résumé en arabe

تُمثِّل الوثائق التقنية تحديًا كبيرًا للمطورين البرمجيين، إذ تستغرق وقتًا طويلاً، وكثيرًا ما يُهمل إعدادها ويصعب الحفاظ عليها. يُقدِّم هذا المشروع منصة **DocuSense**، وهي منصة ويب ذكية تعمل على أتمتة إنشاء أدلة المستخدمين بالاستعانة بالذكاء الاصطناعي التوليدي.

تستعين **DocuSense** بواجهة برمجة تطبيقات **Flash 2.0 Gemini Google** لتحليل لقطات الشاشة ومقاطع الفيديو لواجهات التطبيقات، وتوليد خطوات التوثيق المنظمة تلقائيًا. تتضمن المنصة نظام تلقيب مؤلفًا من اختبارات تفاعلية وشارات تقدم (برونز وفضي وذهبي) وقائمة متصدرين تنافسية، بهدف تعزيز تفاعل المستخدمين وتحسين استيعابهم للمعرفة.

تقنيًا، يعتمد التطبيق على **React.js** للواجهة الأمامية، و**Node.js/Express.js** للخلفية، و**MySQL** عبر **Sequelize ORM** لتخزين البيانات، والمصادقة الآمنة بواسطة **JWT**.
الكلمات المفتاحية: توثيق تلقائي، ذكاء اصطناعي توليدي، تلقيب، Gemini، AI، JWT.

Table des matières

Remerciements	iii
Résumé	iv
Liste des figures	ix
Liste des tableaux	x
Liste des abréviations	xi
Introduction générale	1
1 État de l’art et étude de l’existant	3
1.1 Introduction	3
1.2 La documentation technique logicielle	3
1.2.1 Définition et importance	3
1.2.2 Les défis de la documentation manuelle	4
1.3 L’intelligence artificielle générative	4
1.3.1 Définition et évolution	4
1.3.2 Les modèles multimodaux	4
1.3.3 Google Gemini 2.0 Flash	5
1.4 La gamification	5
1.4.1 Concept et principes	5
1.4.2 Impact de la gamification sur l’apprentissage	6
1.5 Étude de l’existant	6
1.5.1 Solutions actuelles	6
1.5.2 Positionnement de DocuSense	6
1.6 Technologies retenues	6
1.6.1 Architecture générale	6
1.6.2 Frontend – React.js	7
1.6.3 Backend – Node.js / Express.js	7
1.6.4 Base de données – MySQL / Sequelize	7
1.6.5 Authentification – JWT / bcrypt	7
1.6.6 Traitement multimédia – Multer / Sharp	8
1.7 Conclusion	8

2	Analyse et conception	9
2.1	Introduction	9
2.2	Identification des acteurs	9
2.3	Spécification des besoins	9
2.3.1	Besoins fonctionnels	9
2.3.1.1	Module Authentification	9
2.3.1.2	Module Gestion des manuels	10
2.3.1.3	Module Intelligence Artificielle	10
2.3.1.4	Module Gamification	10
2.3.2	Besoins non fonctionnels	11
2.4	Modélisation UML	11
2.4.1	Diagramme des cas d'utilisation	11
2.4.2	Diagramme de classes	11
2.4.3	Diagramme de séquence – Analyse d'image par IA	13
2.5	Conception de la base de données	13
2.5.1	Schéma relationnel	13
2.5.2	Relations entre les tables	13
2.6	Conception de l'API REST	14
2.6.1	Endpoints de l'API	14
2.6.2	Middleware d'authentification JWT	14
2.7	Conclusion	15
3	Réalisation et tests	16
3.1	Introduction	16
3.2	Environnement de développement	16
3.2.1	Outils et versions	16
3.2.2	Structure du projet	16
3.3	Implémentation du backend	17
3.3.1	Configuration du serveur	17
3.3.2	Modèles Sequelize	18
3.3.2.1	Modèle User	18
3.3.2.2	Modèle Progress (gamification)	19
3.3.3	Module Intelligence Artificielle	19
3.3.4	Module Gamification	20
3.4	Implémentation du frontend	20
3.4.1	Configuration du client Axios	21
3.4.2	Routeur principal React	21
3.5	Présentation des interfaces	21
3.5.1	Page d'accueil (Landing)	22

3.5.2	Pages d'authentification	22
3.5.3	Tableau de bord (Dashboard)	22
3.5.3.1	Sidebar de navigation	22
3.5.3.2	Zone principale	23
3.5.4	Module Gamification (Quiz)	23
3.6	Tests	23
3.6.1	Tests des routes API	23
3.6.2	Tests fonctionnels	24
3.6.3	Gestion des erreurs	24
3.7	Conclusion	24
Conclusion générale et perspectives		25
Bibliographie		27
A Variables d'environnement		29
B Instructions de déploiement		30
C Exemple de réponse JSON de l'API Gemini		31
D Glossaire		32

Table des figures

1.1	Architecture trois-tiers de DocuSense	7
2.1	Diagramme des cas d'utilisation de DocuSense	11
2.2	Diagramme de classes de DocuSense	12
2.3	Diagramme de séquence – Analyse d'image par IA	13

Liste des tableaux

1.1	Défis de la documentation technique manuelle	4
1.2	Caractéristiques de Google Gemini 2.0 Flash	5
1.3	Analyse comparative des outils de documentation existants	6
2.1	Acteurs du système DocuSense	9
2.2	Besoins non fonctionnels de DocuSense	11
2.3	Tables de la base de données DocuSense	13
2.4	Tableau des endpoints de l'API DocuSense	14
3.1	Environnement de développement	16
3.2	Résultats des tests API	23
3.3	Tests fonctionnels de DocuSense	24

Liste des abréviations

AI / IA	Artificial Intelligence / Intelligence Artificielle
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SGBDR	Système de Gestion de Base de Données Relationnelle
SPA	Single Page Application
SQL	Structured Query Language
UX	User Experience

Introduction générale

Contexte

Dans l'écosystème numérique contemporain, la documentation technique occupe une place centrale dans la réussite d'un produit logiciel. Elle constitue le pont entre les développeurs qui créent un logiciel et les utilisateurs finaux qui l'exploitent au quotidien. Pourtant, la réalité du terrain montre que cette documentation est trop souvent sacrifiée sur l'autel des délais de livraison : complexe à maintenir, chronophage à rédiger, elle est reléguée au second plan dans la plupart des cycles de développement.

Une étude menée par la société *Stack Overflow* auprès de développeurs révèle que la documentation insuffisante ou inexistante est l'une des principales sources de frustration dans les projets logiciels. Face à ce constat, l'intelligence artificielle générative ouvre de nouvelles perspectives : des modèles de langage multimodaux, capables de comprendre à la fois du texte et des images, peuvent désormais analyser automatiquement une interface graphique et en extraire une documentation structurée.

Problématique

Comment permettre à un développeur de générer en quelques minutes une documentation utilisateur complète, interactive et engageante, à partir d'une simple capture d'écran ou d'une vidéo de son application ?

Objectifs

Ce projet de fin d'études vise à concevoir et développer **DocuSense**, une plateforme web qui :

1. Automatise la génération de documentation technique via l'IA générative multimodale ;
2. Intègre un système de gamification (quiz, badges, classement) pour améliorer l'engagement des utilisateurs ;
3. Offre une interface moderne, intuitive et responsive ;
4. Garantit la sécurité des données via une authentification par JWT.

Structure du rapport

Ce rapport est organisé en trois chapitres :

- **Chapitre 1 – État de l’art et étude de l’existant** : Présentation du domaine, analyse des outils existants et justification des choix technologiques.
- **Chapitre 2 – Analyse et conception** : Spécification des besoins, modélisation UML, conception de l’architecture et de la base de données.
- **Chapitre 3 – Réalisation et tests** : Présentation de l’environnement de développement, implémentation des fonctionnalités clés et des interfaces.

Déclaration sur l’usage de l’Intelligence Artificielle

Conformément à la Charte de rédaction de l’IFAG (§ 2.18), nous déclarons explicitement que des outils d’intelligence artificielle générative ont été utilisés dans le cadre de ce projet :

- **Google Gemini 2.0 Flash** constitue le cœur fonctionnel du moteur d’analyse de DocuSense ; son usage est l’objet même du projet.
- Des assistants IA (*Claude*, *ChatGPT*) ont été ponctuellement employés comme aide au débogage et à la rédaction de certains passages.

L’ensemble du code a été intégralement compris, testé et validé par l’étudiante. Tout contenu généré par IA a été relus, reformulé et contextualisé.

État de l’art et étude de l’existant

1.1 Introduction

La conception d’une plateforme intelligente de génération de documentation nécessite une maîtrise solide du domaine dans lequel elle s’inscrit. Avant d’aborder l’implémentation technique de DocuSense, il est indispensable d’établir un socle conceptuel clair et de situer notre contribution par rapport aux travaux et outils existants. Ce chapitre constitue ce fondement théorique et analytique.

Nous débutons par une définition de la documentation technique logicielle, en examinant ses formes, ses enjeux et les limites des approches manuelles traditionnelles. Nous présentons ensuite les avancées récentes en matière d’intelligence artificielle générative et de traitement multimodal, qui constituent le moteur de notre solution. Une étude comparative des outils existants dans ce domaine nous permettra de dégager le positionnement différenciant de DocuSense. Enfin, nous exposons et justifions l’ensemble des technologies retenues pour le développement de la plateforme.

1.2 La documentation technique logicielle

1.2.1 Définition et importance

Documentation technique

La documentation technique logicielle désigne l’ensemble des artefacts écrits ou multimédias qui décrivent l’utilisation, la configuration, l’architecture et les fonctionnalités d’un système informatique. Elle se décline en deux catégories principales : la documentation destinée aux développeurs (API docs, code docs) et la documentation destinée aux utilisateurs finaux (manuels utilisateurs, guides de démarrage).

La documentation utilisateur remplit plusieurs fonctions essentielles :

- **Réduction du support technique :** Une documentation claire diminue significativement le volume de tickets d’assistance.

- **Accélération de l'adoption** : Les utilisateurs autonomes adoptent plus rapidement un produit.
- **Amélioration de l'expérience utilisateur** : Une bonne documentation réduit la courbe d'apprentissage.
- **Pérennité du produit** : Elle facilite la maintenance et l'évolution du logiciel.

1.2.2 Les défis de la documentation manuelle

La rédaction manuelle de documentation présente plusieurs inconvénients majeurs :

TAB. 1.1 : Défis de la documentation technique manuelle

Défi	Impact
Temps de rédaction élevé	20 à 40 % du temps de développement peut être consacré à la documentation
Obsolescence rapide	Chaque mise à jour du logiciel nécessite une mise à jour manuelle
Qualité variable	Dépend fortement des compétences rédactionnelles du développeur
Manque d'engagement	Les docs statiques génèrent peu d'interaction avec l'utilisateur
Coût élevé	Nécessite parfois des équipes spécialisées (technical writers)

1.3 L'intelligence artificielle générative

1.3.1 Définition et évolution

IA générative

L'intelligence artificielle générative désigne un ensemble de modèles d'apprentissage automatique capables de produire du contenu nouveau (texte, images, code, audio) à partir de données d'entraînement massives. Les grands modèles de langage (LLM) et les modèles multimodaux constituent les piliers de cette révolution technologique.

L'avènement des LLM (GPT-4, Gemini, Claude, LLaMA) a ouvert des perspectives inédites pour l'automatisation de tâches cognitives complexes, dont la rédaction de documentation technique.

1.3.2 Les modèles multimodaux

Les modèles multimodaux étendent les capacités des LLM en intégrant la compréhension d'images, de vidéos et d'audio en plus du texte. **Google Gemini**, utilisé dans DocuSense, est

l'un des premiers modèles multimodaux de grande échelle capables de :

- Analyser une capture d'écran d'interface et en décrire le fonctionnement ;
- Extraire des informations structurées d'une vidéo de démonstration ;
- Générer du contenu textuel formaté (JSON, Markdown) à partir d'entrées mixtes.

1.3.3 Google Gemini 2.0 Flash

Le modèle **Gemini 2.0 Flash** retenu pour DocuSense présente les caractéristiques suivantes :

TAB. 1.2 : Caractéristiques de Google Gemini 2.0 Flash

Caractéristique	Valeur
Type	Grand modèle de langage multimodal
Modalités	Texte, image, vidéo, audio
Vitesse	Très rapide (optimisé pour les applications en temps réel)
Format de sortie	Texte structuré, JSON, Markdown
Fenêtre de contexte	1 million de tokens
Accès	Via API REST (Google AI Studio)
Tarification	Freemium avec quota gratuit

1.4 La gamification

1.4.1 Concept et principes

Gamification

La gamification (ou ludification) est l'application de mécanismes et d'éléments de jeu dans des contextes non-ludiques, dans le but d'augmenter l'engagement, la motivation et la rétention des utilisateurs. Elle s'appuie sur la psychologie comportementale, notamment la théorie de l'autodétermination et le circuit de récompense.

Les éléments de gamification les plus couramment utilisés sont :

- **Points** : Récompense numérique pour chaque action accomplie.
- **Badges** : Distinctions visuelles attribuées lors d'étapes clés.
- **Classement** : Tableau de comparaison des performances entre utilisateurs.
- **Quiz** : Évaluation des connaissances avec feedback immédiat.

- **Progression** : Indicateur visuel de l'avancement dans un parcours.

1.4.2 Impact de la gamification sur l'apprentissage

Des études montrent que la gamification augmente de 40 à 60 % l'engagement des utilisateurs dans les plateformes d'apprentissage en ligne. Dans le contexte de la documentation logicielle, elle transforme une lecture passive en une expérience active et mémorable.

1.5 Étude de l'existant

1.5.1 Solutions actuelles

TAB. 1.3 : Analyse comparative des outils de documentation existants

Outil	IA	Multimodal	Gamif.	Gratuit	Limitation principale
Notion	Oui	Non	Non	Partiel	Pas d'analyse d'interface
Confluence	Part.	Non	Non	Non	Coût élevé, complexité
GitBook	Oui	Non	Non	Partiel	Pas d'IA multimodale
Scribe	Oui	Oui	Non	Partiel	Pas de gamification
Document360	Oui	Non	Non	Non	Coût prohibitif
DocuSense	Oui	Oui	Oui	Oui	Notre solution

1.5.2 Positionnement de DocuSense

DocuSense se distingue de l'existant par la combinaison unique de trois capacités :

1. **Génération IA multimodale** : Analyse de screenshots *et* de vidéos.
2. **Gamification intégrée** : Quiz, badges et classement dans la même plateforme.
3. **Accessibilité** : Gratuit, sans installation, utilisable dès la création du compte.

1.6 Technologies retenues

1.6.1 Architecture générale

L'application DocuSense repose sur une architecture **client-serveur** à trois niveaux (three-tier) :

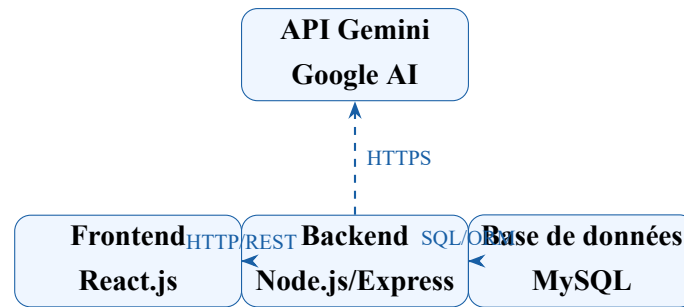


FIG. 1.1 : Architecture trois-tiers de DocuSense

1.6.2 Frontend – React.js

React.js (v19) est une bibliothèque JavaScript développée par Meta pour la construction d'interfaces utilisateur. Ses avantages principaux pour DocuSense sont :

- Composants réutilisables et maintenance simplifiée ;
- Rendu déclaratif et gestion d'état via `useState/useEffect` ;
- Écosystème riche (React Router, Axios, Ant Design) ;
- Performances optimisées grâce au DOM virtuel.

1.6.3 Backend – Node.js / Express.js

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Chrome. Couplé au framework **Express.js** (v5), il offre :

- Une architecture non bloquante et événementielle adaptée aux I/O intensives ;
- La même syntaxe JavaScript que le frontend (cohérence de la stack) ;
- Une API REST structurée avec routage modulaire.

1.6.4 Base de données – MySQL / Sequelize

MySQL est un SGBDR open-source largement adopté. L'ORM **Sequelize** simplifie les interactions en permettant la définition des modèles en JavaScript et la synchronisation automatique du schéma.

1.6.5 Authentification – JWT / bcrypt

La sécurité des endpoints est assurée par :

- **bcryptjs** : Hachage des mots de passe avec salt aléatoire ;
- **jsonwebtoken (JWT)** : Tokens d'authentification stateless avec expiration configurable (30 jours).

1.6.6 Traitement multimédia – Multer / Sharp

- **Multer** : Middleware de gestion des uploads de fichiers (images, vidéos) avec stockage sur disque ;
- **Sharp** : Bibliothèque de traitement d'images haute performance pour la compression (re-size 1280×720, qualité JPEG 70 %) avant envoi à l'API Gemini.

1.7 Conclusion

Ce chapitre a permis de poser le cadre conceptuel et technologique de DocuSense. L'analyse de l'existant confirme l'originalité de notre approche combinant IA multimodale et gamification. Les technologies retenues forment un écosystème cohérent, moderne et adapté aux contraintes du projet.

Analyse et conception

2.1 Introduction

Ce chapitre présente l'analyse fonctionnelle et technique de DocuSense. Nous identifions les acteurs, spécifions les besoins fonctionnels et non fonctionnels, puis concevons l'architecture logicielle et le schéma de la base de données.

2.2 Identification des acteurs

L'application DocuSense implique deux types d'acteurs principaux :

TAB. 2.1 : Acteurs du système DocuSense

Acteur	Rôle	Actions principales
Développeur	Créateur de contenu	S'inscrire, créer des manuels, utiliser l'IA, publier des manuels
Utilisateur final	Consommateur de contenu	Consulter les manuels, répondre aux quiz, gagner des badges
Système IA	Acteur externe	Analyser les entrées (texte, image, vidéo) et retourner des étapes JSON

2.3 Spécification des besoins

2.3.1 Besoins fonctionnels

2.3.1.1 Module Authentification

- **BF-01** : L'utilisateur peut créer un compte avec nom, email et mot de passe.
- **BF-02** : L'utilisateur peut se connecter avec email et mot de passe.
- **BF-03** : Le système génère un token JWT valide 30 jours lors de la connexion.
- **BF-04** : L'utilisateur peut se déconnecter (suppression du token côté client).

- **BF-05** : Toutes les routes protégées vérifient la validité du token.

2.3.1.2 *Module Gestion des manuels*

- **BF-06** : Le développeur peut créer un manuel avec titre et description.
- **BF-07** : Le développeur peut consulter la liste de ses manuels.
- **BF-08** : Le développeur peut modifier ou supprimer un manuel.
- **BF-09** : Le développeur peut publier ou dépublier un manuel.
- **BF-10** : Chaque manuel est lié à son auteur.

2.3.1.3 *Module Intelligence Artificielle*

- **BF-11** : Le développeur peut soumettre une description textuelle pour générer 5 étapes de documentation.
- **BF-12** : Le développeur peut uploader une image (screenshot) pour générer 5 étapes via l'analyse visuelle de l'IA.
- **BF-13** : Le développeur peut uploader une vidéo pour générer 5 étapes via l'analyse vidéo de l'IA.
- **BF-14** : Les étapes générées sont retournées au format JSON structuré.
- **BF-15** : Les fichiers sont compressés avant envoi à l'API (optimisation).

2.3.1.4 *Module Gamification*

- **BF-16** : Le système associe un quiz à chaque étape.
- **BF-17** : L'utilisateur répond à un quiz et obtient un feedback immédiat (correct/incorrect).
- **BF-18** : Les points sont cumulés et stockés dans le profil de progression.
- **BF-19** : Les badges sont attribués automatiquement selon les seuils : Bronze (50 pts), Silver (100 pts), Gold (200 pts).
- **BF-20** : Un classement affiche les 10 meilleurs scores.

2.3.2 Besoins non fonctionnels

TAB. 2.2 : Besoins non fonctionnels de DocuSense

ID	Catégorie	Description
BNF-01	Sécurité	Les mots de passe sont hachés (bcrypt, salt=10).
BNF-02	Sécurité	Toutes les communications utilisent des tokens JWT signés.
BNF-03	Performance	Compression des images avant envoi à l'API (Sharp).
BNF-04	Performance	Réponse de l'API IA en moins de 10 secondes.
BNF-05	Disponibilité	L'application est accessible 24h/24 via navigateur.
BNF-06	Ergonomie	Interface responsive, palette de couleurs cohérente.
BNF-07	Maintenabilité	Architecture modulaire (contrôleurs, modèles, routes séparés).
BNF-08	Extensibilité	Ajout facile de nouveaux types d'étapes (text, video, quiz).

2.4 Modélisation UML

2.4.1 Diagramme des cas d'utilisation

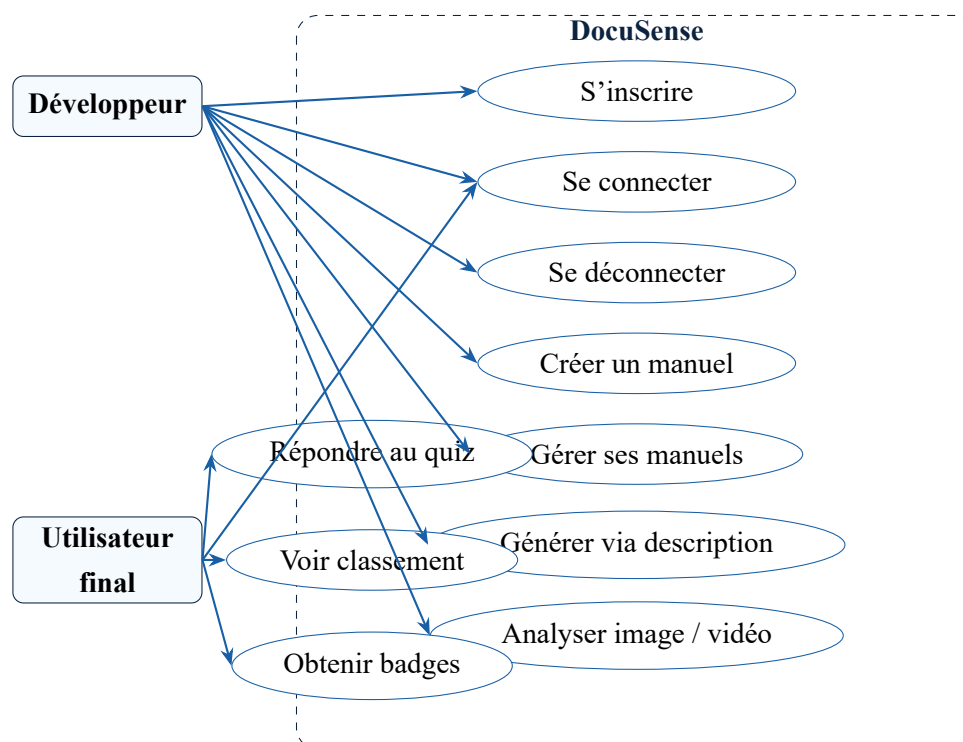


FIG. 2.1 : Diagramme des cas d'utilisation de DocuSense

2.4.2 Diagramme de classes

Le diagramme de classes ci-dessous représente le modèle de données de DocuSense tel qu'il est implémenté avec Sequelize :

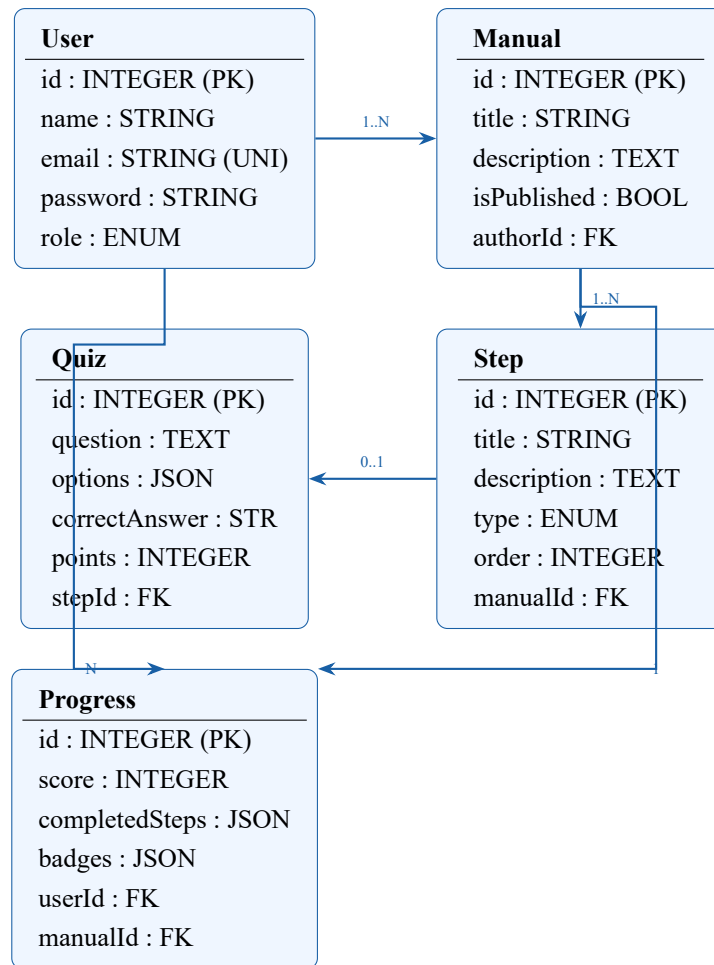


FIG. 2.2 : Diagramme de classes de DocuSense

2.4.3 Diagramme de séquence – Analyse d'image par IA

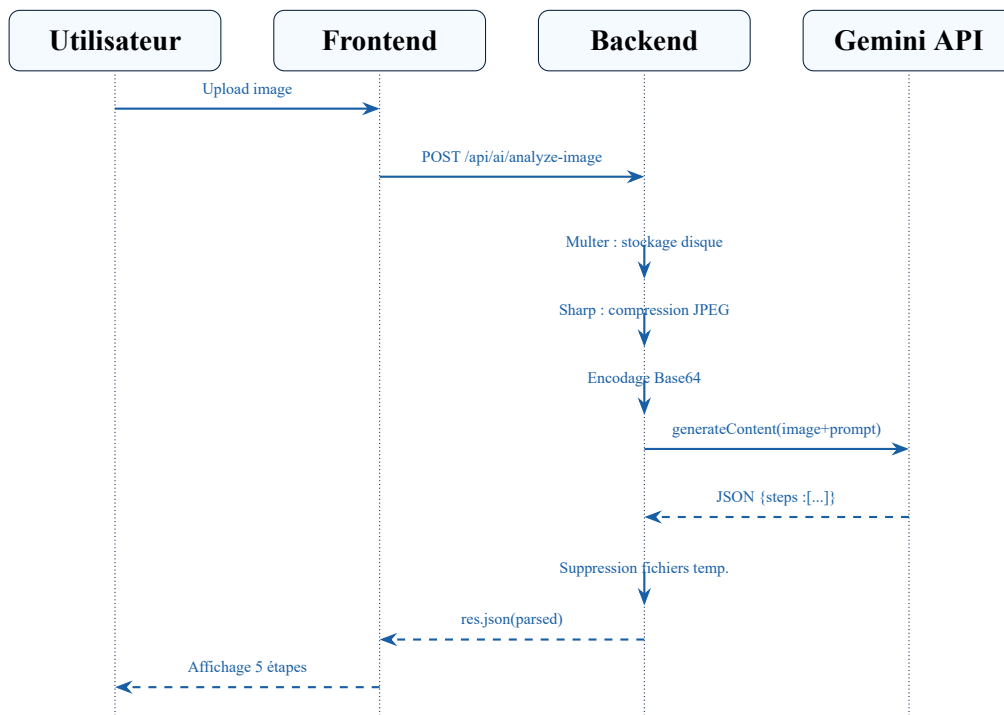


FIG. 2.3 : Diagramme de séquence – Analyse d'image par IA

2.5 Conception de la base de données

2.5.1 Schéma relationnel

La base de données MySQL de DocuSense comporte cinq tables :

TAB. 2.3 : Tables de la base de données DocuSense

Table	Clé primaire	Description
Users	id (AI)	Comptes utilisateurs
Manuals	id (AI)	Manuels créés par les développeurs
Steps	id (AI)	Étapes de documentation
Quizzes	id (AI)	Questions d'évaluation associées à une étape
Progresses	id (AI)	Suivi de progression par (utilisateur, manuel)

2.5.2 Relations entre les tables

- Users → Manuals : Un utilisateur peut créer plusieurs manuels (1..N).
- Manuals → Steps : Un manuel contient plusieurs étapes (1..N).
- Steps → Quizzes : Une étape peut avoir un quiz (0..1).

- Users + Manuals → Progresses : Suivi par couple (utilisateur, manuel).

2.6 Conception de l'API REST

2.6.1 Endpoints de l'API

TAB. 2.4 : Tableau des endpoints de l'API DocuSense

Méthode	Route	Auth.	Description
POST	/api/auth/register	Non	Inscription
POST	/api/auth/login	Non	Connexion, retourne JWT
GET	/api/manuals	Oui	Liste des manuels
POST	/api/manuals	Oui	Créer un manuel
GET	/api/manuals/:id	Oui	Manuel + étapes
PUT	/api/manuals/:id	Oui	Modifier un manuel
DELETE	/api/manuals/:id	Oui	Supprimer un manuel
POST	/api/ai/suggest	Oui	Générer étapes (texte)
POST	/api/ai/analyze-image	Oui	Générer étapes (image)
POST	/api/ai/analyze-video	Oui	Générer étapes (vidéo)
POST	/api/quiz/create	Oui	Créer un quiz
POST	/api/quiz/submit	Oui	Soumettre une réponse
GET	/api/quiz/leaderboard	Oui	Top 10 classement

2.6.2 Middleware d'authentification JWT

Le middleware de protection vérifie le token JWT sur toutes les routes sécurisées :

```

1 const protect = (req, res, next) => {
2   const token = req.headers.authorization?.split(' ')[1]
3   if (!token) {
4     return res.status(401).json({ message: 'Non autorise, token manquant' })
5   }
6   try {
7     const decoded = jwt.verify(token, process.env.JWT_SECRET)
8     req.user = decoded
9     next()

```

```
10   } catch (err) {  
11     res.status(401).json({ message: 'Token invalide' })  
12   }  
13 }
```

Listing 2.1: Middleware d’authentification (middleware/authMiddleware.js)

2.7 Conclusion

Ce chapitre a présenté l’analyse complète des besoins et la conception architecturale de DocuSense. Les diagrammes UML, le schéma de base de données et la spécification de l’API forment une base solide pour la phase de réalisation.

Réalisation et tests

3.1 Introduction

Ce chapitre présente l'environnement de développement, les aspects techniques clés de l'implémentation, et les interfaces graphiques de l'application DocuSense.

3.2 Environnement de développement

3.2.1 Outils et versions

TAB. 3.1 : Environnement de développement

Technologie	Version	Rôle
Node.js	20.x	Environnement d'exécution serveur
React.js	19.2.5	Framework frontend
Express.js	5.2.1	Framework backend
Sequelize	6.37.8	ORM pour MySQL
MySQL	8.x	Base de données relationnelle
@google/generative-ai	0.24.1	SDK Gemini
Multer	2.1.1	Upload de fichiers
Sharp	0.34.5	Traitement d'images
bcryptjs	3.0.3	Hachage de mots de passe
jsonwebtoken	9.0.3	Authentification JWT
Ant Design	6.3.7	Bibliothèque UI
Axios	1.16.0	Client HTTP
React Router	7.14.2	Routage SPA
VS Code	–	Éditeur de code
Postman	–	Test des API REST

3.2.2 Structure du projet

```
1 docusense/
2 |-- backend/
3 |   |-- config/
4 |   |   `-- db.js          # Connexion MySQL via Sequelize
5 |   |-- controllers/
6 |   |   |-- aiController.js  # Logique IA (Gemini)
7 |   |   |-- authController.js # Inscription / Connexion
8 |   |   |-- manualController.js # CRUD manuels
9 |   |   `-- quizController.js # Quiz et gamification
10 |   |-- middleware/
11 |   |   `-- authMiddleware.js # Verification JWT
12 |   |-- models/
13 |   |   |-- User.js
14 |   |   |-- Manual.js
15 |   |   |-- Step.js
16 |   |   |-- Quiz.js
17 |   |   `-- Progress.js
18 |   |-- routes/
19 |   |   |-- authRoutes.js
20 |   |   |-- manualRoutes.js
21 |   |   |-- aiRoutes.js
22 |   |   `-- quizRoutes.js
23 |   |-- .env
24 |   |-- package.json
25 |   `-- server.js
26 `-- frontend2/
27     `-- src/
28         |-- pages/
29         |   |-- Landing.js
30         |   |-- Login.js
31         |   |-- Register.js
32         |   |-- Dashboard.js
33         |   `-- Quiz.js
34         |-- services/
35         |   `-- api.js
36         `-- App.js
```

Listing 3.1: Arborescence du projet DocuSense

3.3 Implémentation du backend

3.3.1 Configuration du serveur

Le fichier `server.js` constitue le point d'entrée de l'application. Il configure Express, les middlewares CORS, JSON, importe les modèles Sequelize et monte les routes :

```
1 const express = require('express')
2 const dotenv = require('dotenv')
3 const cors = require('cors')
4 const { connectDB, sequelize } = require('./config/db')
5
6 dotenv.config()
7 const app = express()
8
9 app.use(cors({ origin: 'http://localhost:3000', credentials: true }))
10 app.use(express.json())
11
12 require('./models/User')
13 require('./models/Manual')
14 require('./models/Step')
15 require('./models/Quiz')
16 require('./models/Progress')
17
18 app.use('/api/auth', require('./routes/authRoutes'))
19 app.use('/api/manuals', require('./routes/manualRoutes'))
20 app.use('/api/ai', require('./routes/aiRoutes'))
21 app.use('/api/quiz', require('./routes/quizRoutes'))
22
23 const start = async () => {
24   await connectDB()
25   await sequelize.sync({ alter: true }) // Sync automatique du schema
26   app.listen(process.env.PORT || 5000)
27 }
28 start()
```

Listing 3.2: Configuration du serveur Express (server.js)

3.3.2 Modèles Sequelize

3.3.2.1 Modèle User

```
1 const User = sequelize.define('User', {
2   id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
3   name: { type: DataTypes.STRING, allowNull: false },
4   email: { type: DataTypes.STRING, allowNull: false, unique: true },
5   password: { type: DataTypes.STRING, allowNull: false },
6   role: { type: DataTypes.ENUM('developer', 'enduser'),
7     defaultValue: 'developer' },
8 })
```

Listing 3.3: Modèle User (models/User.js)

3.3.2.2 Modèle Progress (gamification)

```
1 const Progress = sequelize.define('Progress', {
2   score:          { type: DataTypes.INTEGER, defaultValue: 0 },
3   completedSteps: { type: DataTypes.JSON,    defaultValue: [] },
4   isCompleted:    { type: DataTypes.BOOLEAN, defaultValue: false },
5   badges:         { type: DataTypes.JSON,    defaultValue: [] },
6 })
7 Progress.belongsTo(User, { foreignKey: 'userId' })
8 Progress.belongsTo(Manual, { foreignKey: 'manualId' })
```

Listing 3.4: Modèle Progress (models/Progress.js)

3.3.3 Module Intelligence Artificielle

Le contrôleur IA regroupe trois fonctions. Voici l'implémentation de l'analyse d'image qui est la fonctionnalité centrale :

```
1 const analyzeImage = async (req, res) => {
2   if (!req.file)
3     return res.status(400).json({ message: 'Aucune image fournie' })
4
5   const model = genAI.getGenerativeModel({ model: 'gemini-2.0-flash' })
6
7   // Compression de l'image via Sharp
8   const compressedPath = req.file.path + '_compressed.jpg'
9   await sharp(req.file.path)
10    .resize(1280, 720, { fit: 'inside' })
11    .jpeg({ quality: 70 })
12    .toFile(compressedPath)
13
14   const base64Image = fs.readFileSync(compressedPath).toString('base64')
15
16   const result = await model.generateContent([
17     { inlineData: { data: base64Image, mimeType: 'image/jpeg' } },
18     `Tu es un expert en documentation technique UX.
19     Analyse cette capture d'ecran et genere exactement 5 etapes.
20     Reponds UNIQUEMENT en JSON :
21     {"steps":[{"title":"...", "description":"...", "type":"text"},...]}`,
22   ])
23
24   // Nettoyage des fichiers temporaires
25   fs.unlinkSync(req.file.path)
26   fs.unlinkSync(compressedPath)
27
28   const text = result.response.text().replace(/``json|``/g, '').trim()
29 }
```

```
29   res.json(JSON.parse(text))
30 }
```

Listing 3.5: Analyse d'image par Gemini (controllers/aiController.js)

3.3.4 Module Gamification

La logique d'attribution des badges est intégrée dans le contrôleur de quiz :

```
1  const submitAnswer = async (req, res) => {
2    const quiz = await Quiz.findByPk(req.body.quizId)
3    const isCorrect = quiz.correctAnswer === req.body.answer
4    const pointsEarned = isCorrect ? quiz.points : 0
5
6    let progress = await Progress.findOne({
7      where: { userId: req.user.id, manualId: req.body.manualId }
8    })
9    if (!progress) {
10     progress = await Progress.create({
11       userId: req.user.id, manualId: req.body.manualId,
12       score: 0, completedSteps: [], badges: []
13     })
14   }
15
16   const newScore = progress.score + pointsEarned
17   let badges = [...(progress.badges || [])]
18
19   // Attribution automatique des badges par seuils
20   if (newScore >= 50 && !badges.includes('bronze')) badges.push('bronze')
21   if (newScore >= 100 && !badges.includes('silver')) badges.push('silver')
22   if (newScore >= 200 && !badges.includes('gold')) badges.push('gold')
23
24   await progress.update({ score: newScore, badges,
25     completedSteps: [...(progress.completedSteps || []), req.body.quizId] })
26
27   res.json({ isCorrect, pointsEarned, totalScore: newScore, badges })
28 }
```

Listing 3.6: Attribution des badges (controllers/quizController.js)

3.4 Implémentation du frontend

3.4.1 Configuration du client Axios

Le service API centralise toutes les requêtes HTTP et injecte automatiquement le token JWT :

```
1 import axios from 'axios'
2
3 const API = axios.create({ baseURL: 'http://localhost:5000/api' })
4
5 API.interceptors.request.use((req) => {
6   const token = localStorage.getItem('token')
7   if (token) req.headers.Authorization = `Bearer ${token}`
8   return req
9 })
10
11 export default API
```

Listing 3.7: Client Axios (services/api.js)

3.4.2 Routeur principal React

```
1 const App = () => {
2   const token = localStorage.getItem('token')
3   return (
4     <BrowserRouter>
5       <Routes>
6         <Route path="/" element={<Landing />} />
7         <Route path="/login" element={<Login />} />
8         <Route path="/register" element={<Register />} />
9         <Route path="/dashboard"
10           element={token ? <Dashboard /> : <Navigate to="/login" />} />
11         <Route path="/quiz"
12           element={token ? <Quiz /> : <Navigate to="/login" />} />
13       </Routes>
14     </BrowserRouter>
15   )
16 }
```

Listing 3.8: Routeur React (App.js)

3.5 Présentation des interfaces

3.5.1 Page d'accueil (Landing)

La page d'accueil de DocuSense présente les fonctionnalités de la plateforme à travers une interface marketing moderne. Elle comprend :

- Une **navbar** avec les liens vers Connexion et Inscription ;
- Une **section Hero** avec le slogan, la description et un aperçu d'un manuel généré par IA ;
- Des **statistiques** clés : 100 % automatisé, 5 min pour créer un manuel, 3× plus d'engagement, gratuit ;
- Une **grille de fonctionnalités** (IA générative, Gamification, Ultra rapide) ;
- Un **call-to-action** de fin de page.

Interface Landing

La palette de couleurs (bleu #185FA5, bleu foncé #0C2340, violet #8E44AD) est cohérente sur toute l'application et reflète un positionnement professionnel et technologique.

3.5.2 Pages d'authentification

Les pages **Login** et **Register** adoptent un design épuré avec :

- Fond bleu clair (#E6F1FB) ;
- Carte blanche centrée avec ombre portée ;
- Champs de saisie stylisés avec bord blue ;
- Feedback d'erreur en rouge en cas d'échec.

La connexion stocke le JWT dans `localStorage` et redirige vers le dashboard.

3.5.3 Tableau de bord (Dashboard)

Le dashboard est l'interface principale des développeurs, organisé en :

3.5.3.1 Sidebar de navigation

- Logo DocuSense avec dégradé bleu ;
- Menu principal : Dashboard, Mes manuels, Module IA, Gamification ;
- Bouton Déconnexion en rouge.

3.5.3.2 Zone principale

- **Topbar** avec titre et bouton « Nouveau manuel » ;
- **Métriques en cartes** : Total manuels, Publiés, Score total ;
- **Grille de manuels** avec pour chaque manuel :
 - Titre, description et statut (brouillon/publié) ;
 - Champ de description pour génération textuelle ;
 - Bouton « Suggérer par description » ;
 - Boutons d'upload image et vidéo ;
 - Affichage des étapes suggérées par l'IA.

3.5.4 Module Gamification (Quiz)

La page Quiz présente le système de progression :

- **Classement** : Tableau des 10 meilleurs scores avec positions ;
- **Badges** : Affichage visuel des trois badges :
 - **Bronze** (50 pts), **Silver** (100 pts), **Gold** (200 pts) ;
- **Résultat de quiz** : Feedback coloré (vert si correct, rouge si incorrect) avec points gagnés et badges obtenus.

3.6 Tests

3.6.1 Tests des routes API

Les endpoints ont été testés avec **Postman**. Le tableau suivant résume les résultats :

TAB. 3.2 : Résultats des tests API

Endpoint	Méthode	Statut	Résultat
/api/auth/register	POST	201	Compte créé, JWT retourné
/api/auth/login	POST	200	Authentification, JWT retourné
/api/manuals	GET	200	Liste des manuels
/api/manuals	POST	201	Manuel créé
/api/ai/suggest	POST	200	5 étapes JSON générées
/api/ai/analyze-image	POST	200	5 étapes depuis screenshot
/api/quiz/submit	POST	200	Score mis à jour, badge
/api/quiz/leaderboard	GET	200	Top 10 retourné

3.6.2 Tests fonctionnels

TAB. 3.3 : Tests fonctionnels de DocuSense

Scénario	Description	Résultat
Inscription	Création d'un compte valide	OK
Connexion	Login avec identifiants corrects	OK
Token invalide	Accès route protégée sans JWT	401
Création manuel	Ajout d'un nouveau manuel	OK
Génération texte	Description vers 5 étapes	OK
Analyse image	Screenshot PNG vers 5 étapes	OK
Analyse vidéo	Vidéo MP4 vers 5 étapes	OK
Badge Bronze	Score ≥ 50 pts	OK
Classement	Top 10 trié par score DESC	OK
Déconnexion	Suppression du token localStorage	OK

3.6.3 Gestion des erreurs

DocuSense implémente une gestion des erreurs à plusieurs niveaux :

- **Niveau API** : Codes HTTP appropriés (400, 401, 404, 500) avec messages explicites.
- **Niveau IA** : Try/catch sur tous les appels Gemini avec log des erreurs.
- **Niveau Frontend** : Redirection vers /login si le token est absent ou expiré.
- **Niveau fichiers** : Suppression systématique des fichiers temporaires après traitement.

3.7 Conclusion

Ce chapitre a présenté l'implémentation complète de DocuSense : de la configuration du serveur Express aux modèles Sequelize, en passant par l'intégration de l'API Gemini, le système de gamification et les interfaces React. Les tests réalisés confirment le bon fonctionnement de l'ensemble des fonctionnalités.

Conclusion générale et perspectives

Bilan du projet

Ce projet de fin d'études a abouti au développement d'une application web fonctionnelle : **DocuSense**, une plateforme intelligente de génération de documentation technique gamifiée. Les objectifs initiaux ont été atteints :

- **Automatisation par l'IA** : L'intégration de l'API Google Gemini 2.0 Flash permet de générer en quelques secondes cinq étapes de documentation structurées à partir d'une description textuelle, d'une capture d'écran ou d'une vidéo d'application.
- **Architecture robuste** : La stack Node.js/Express/MySQL/React, combinée à Sequelize ORM et JWT, fournit une application sécurisée, modulaire et maintenable.
- **Gamification opérationnelle** : Le système de quiz, de badges progressifs et de classement compétitif est pleinement fonctionnel.
- **Interface moderne** : Le design cohérent offre une expérience utilisateur qualitative.

Difficultés rencontrées

- La gestion des fichiers volumineux envoyés à l'API Gemini a nécessité un pipeline de compression (Sharp) pour respecter les limites de l'API.
- Le parsing du JSON retourné par Gemini a demandé un nettoyage des balises Markdown avant désérialisation.
- La synchronisation automatique du schéma Sequelize a parfois provoqué des conflits lors de l'évolution des modèles.

Perspectives d'amélioration

1. **Export multiformat** : Génération des manuels en PDF, HTML et Markdown pour une distribution hors-ligne.
2. **Collaboration en temps réel** : Édition collaborative avec WebSockets (Socket.io).
3. **Versionnement des manuels** : Historique des versions pour tracker l'évolution de la documentation.

4. **Support multilingue** : Génération de documentation en plusieurs langues grâce aux capacités multilingues de Gemini.
5. **Tableau de bord analytique** : Statistiques d'utilisation (taux de complétion, scores moyens, questions les plus échouées).
6. **Application mobile** : Version React Native pour la consultation sur smartphone.
7. **Intégrations tierces** : Webhooks vers Slack, GitHub Pages, ou Confluence pour publication automatique.
8. **Sécurité renforcée** : Ajout du refresh token, rate limiting et protection CSRF.

Mot de fin

DocuSense démontre que l'intelligence artificielle générative peut résoudre des problèmes concrets du quotidien des développeurs. En combinant la puissance des LLM multimodaux avec une expérience utilisateur gamifiée, ce projet ouvre la voie à une nouvelle génération d'outils de documentation : automatiques, interactifs et engageants.

Bibliographie

Bibliographie

- [1] Google DeepMind, *Gemini : A Family of Highly Capable Multimodal Models*, Technical Report, Google, 2023.
- [2] Meta Platforms, *React – A JavaScript library for building user interfaces*, Documentation officielle, 2024. <https://react.dev>
- [3] OpenJS Foundation, *Node.js Documentation*, v20 LTS, 2024. <https://nodejs.org/docs/>
- [4] StrongLoop / OpenJS Foundation, *Express.js*, 2024. <https://expressjs.com>
- [5] Sequelize Contributors, *Sequelize ORM*, Documentation v6, 2024. <https://sequelize.org>
- [6] M. Jones, J. Bradley, N. Sakimura, *RFC 7519 : JSON Web Token (JWT)*, IETF, Mai 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [7] N. Provos, D. Mazieres, *A Future-Adaptable Password Scheme*, USENIX, 1999.
- [8] Lovell Fuller, *Sharp – High performance Node.js image processing*, 2024. <https://sharp.pixelplumbing.com>
- [9] Ant Design Team, *Ant Design*, Documentation v6, Alibaba, 2024. <https://ant.design>
- [10] Deterding S., Dixon D., Khaled R., Nacke L. *From game design elements to gamefulness : defining gamification*. Proc. 15th Academic MindTrek Conference, 2011.
- [11] Vaswani A. et al. *Attention is all you need*. Advances in Neural Information Processing Systems, 30, 2017.
- [12] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, PhD Thesis, University of California, 2000.
- [13] Oracle Corporation, *MySQL 8.0 Reference Manual*, 2024. <https://dev.mysql.com/doc/>
- [14] Multer Contributors, *Multer – Node.js middleware for multipart/form-data*, GitHub, 2024.
- [15] Matt Zabriskie, *Axios – Promise based HTTP client*, 2024. <https://axios-http.com>

Variables d'environnement

Le fichier `.env` à la racine du backend contient les configurations sensibles de l'application :

```
1 # Base de donnees MySQL
2 DB_HOST=localhost
3 DB_NAME=docusense
4 DB_USER=root
5 DB_PASSWORD=votre_mot_de_passe
6
7 # Authentification JWT
8 JWT_SECRET=votre_cle_secrete_tres_longue_et_aleatoire
9
10 # API Google Gemini
11 GEMINI_API_KEY=votre_cle_api_gemini
12
13 # Serveur
14 PORT=5000
```

Listing A.1: Structure du fichier `.env` (backend)

Sécurité

Le fichier `.env` ne doit jamais être versionné dans Git. Il doit être ajouté au fichier `.gitignore` pour éviter l'exposition des clés secrètes.

Instructions de déploiement

Prérequisits

- Node.js v18+ installé
- MySQL 8.x installé et en cours d'exécution
- Compte Google AI Studio pour la clé API Gemini

Installation du backend

```
1 cd backend
2 npm install
3 # Editer les variables dans .env
4 node server.js      # ou : npx nodemon server.js
```

Installation du frontend

```
1 cd frontend2
2 npm install
3 npm start          # Lance sur http://localhost:3000
```

Création de la base de données

```
1 CREATE DATABASE docusense
2 CHARACTER SET utf8mb4
3 COLLATE utf8mb4_unicode_ci;
```

Sequelize créera automatiquement les tables au démarrage du serveur grâce à `sequelize.sync({alter: true})`.

Exemple de réponse JSON de l'API Gemini

```
1 {
2   "steps": [
3     {
4       "title": "Connexion et configuration initiale",
5       "description": "Accédez a la page de connexion et saisissez vos identifiants.",
6       "type": "text"
7     },
8     {
9       "title": "Creation du premier projet",
10      "description": "Cliquez sur '+ Nouveau manuel' pour creer votre premier
11      projet de documentation.",
12      "type": "text"
13    },
14    {
15      "title": "Parametrage des options",
16      "description": "Renseignez le titre et la description, puis cliquez sur
17      'Creer'.",
18      "type": "text"
19    },
20    {
21      "title": "Generation automatique par IA",
22      "description": "Utilisez le champ de description pour generer
23      automatiquement les etapes.",
24      "type": "text"
25    },
26    {
27      "title": "Publication et partage",
28      "description": "Publiez votre manuel pour le rendre accessible aux
29      utilisateurs finaux.",
30      "type": "text"
31    }
32  ]
33 }
```

Listing C.1: Exemple de réponse Gemini

Glossaire

API Application Programming Interface – Interface permettant à deux applications de communiquer via des requêtes standardisées.

Base64

Méthode d'encodage binaire en caractères ASCII, utilisée pour transmettre des images à l'API Gemini.

bcrypt

Algorithme de hachage de mots de passe adaptatif résistant aux attaques par force brute.

CORS

Cross-Origin Resource Sharing – Mécanisme HTTP permettant à un serveur d'autoriser les requêtes depuis d'autres origines.

Gemini

Famille de grands modèles de langage multimodaux développés par Google DeepMind.

JWT

JSON Web Token – Standard ouvert (RFC 7519) pour la création de tokens d'accès signés.

LLM

Large Language Model – Modèle de langage d'IA entraîné sur de très larges corpus de texte.

Middleware

Composant logiciel intermédiaire traitant les requêtes HTTP entre leur réception et leur traitement final.

Multimodal

Qualifie un modèle d'IA capable de traiter simultanément plusieurs types de données (texte, image, vidéo, audio).

ORM

Object-Relational Mapping – Technique permettant de manipuler une base de données relationnelle via des objets du langage de programmation.

REST

Representational State Transfer – Style d’architecture pour les API web utilisant les méthodes HTTP.

Sharp

Bibliothèque Node.js haute performance pour le traitement d’images.

SPA Single Page Application – Application web dont le contenu est mis à jour dynamiquement sans rechargement de la page.

Token

Jeton numérique servant à l’authentification et à l’autorisation dans une application.